UFRGS
UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL

Curso de Especialização

Inteligência Artificial para Engenheiros

Disciplina: **Algoritmos e Programação (APE)**

Aula: 01 - Introdução

IA PARA ENGENHEIROS
UFRGS

Professor: **Raphael Martins Brum**

2026

Sobre o Professor

- Prof. Raphael Martins Brum
  - E-mail: brum@ufrgs.br
  - LinkedIn: https://www.linkedin.com/in/rmbrum
  - Telefone: (51) 3308-4446
- Micro-CV:
  - Graduado em Engenharia de Computação pela UFRGS (2007/2)
    - Experiência em programação para EDA com C++, Perl, C
  - *Master recherche* em Sistemas Microeletrônicos pela Université Montpellier (2011)
  - Doutor em Sistemas Microeletrônicos pela Université Montpellier (2014)
  - Professor Adjunto do Departamento de Engenharia Elétrica, EE/UFRGS desde 2017
  - Docente permanente do PPG de Informática na Educação
    - Experiência em programação com Python para extração de dados acadêmicos

Quem são vocês?

- Qual é a formação de vocês?
  - Ciência da Computação
  - Engenharia de Computação
  - Engenharia Elétrica
  - Outras engenharias-irmãs da Eng. Elétrica (Eletrônica, Automação, Energia, …)
  - Outras engenharias
  - Outras exatas
  - Nenhuma das anteriores
- Vocês conhecem noções de algoritmos?
- Vocês sabem programar em alguma linguagem de programação?
- Vocês sabem programar em Python?

Sobre a Disciplina

- **Título:** Algoritmos e Programação
- **Sigla:** APE
- **Carga horária:** 30 horas (12 encontros de 2h30), das 19h30 às 22h15 (intervalo de 15 min)

- **Objetivos:**
  - Introdução aos conceitos de algoritmos e estruturas de dados
  - Nivelamento dos alunos em preparação às disciplinas subsequentes
  - Compreensão das estruturas básicas da linguagem Python
  - Introdução às funcionalidades das bibliotecas mais empregadas no meio
  - Introdução aos ambientes de desenvolvimento integrados (IDE)

Conteúdo Programático

- Introdução a algoritmos
- Características da linguagem Python 3 e IDEs
- Tipos de dados, variáveis e constantes
- Funções e escopo
- Programação estruturada
- Retorno de funções e recursividade
- Vetores, matrizes, tensores; arquivos
- Programação orientada a objetos
- Instalação e uso de bibliotecas de interesse para IA

Metodologia

- Aulas teóricas expositivas, em formato remoto síncrono (como esta)
- Realização de exercícios (fora do horário da aula)
- Proposição de projetos

Critérios de avaliação

- Realização de projetos curtos ao longo dos doze encontros
- Os projetos serão avaliados pelo professor, que lhes atribuirá grau de 0 a 10
- Nota Final NF = Média aritmética dos projetos desenvolvidos
- Ao término da disciplina, será atribuído um dos seguintes conceitos ao aluno
  - A, para 9 <= NF <= 10
  - B, para 7,5 <= NF < 9
  - C, para 6 <= NF < 7,5
  - D, para 0 <= NF < 6

- A frequência inferior a 75% ensejará a aplicação do conceito FF (falta de frequência), nos termos do Art. 134 do RGU

Bibliografia

- CORMEN, Thomas H. et al. **Introduction to algorithms.** MIT press, 2022.
- SEDGEWICK, Robert; WAYNE, Kevin. **Algorithms.** Addison-wesley professional, 2011.
- GUTTAG, John V. **Introduction to computation and programming using Python: With application to understanding data**. MIT press, 2016.
- DEITEL, Paul. **Python for Programmers.** 2019.
- 30-Days-of-Python: https://github.com/Asabeneh/30-Days-Of-Python
- E outras que serão recomendadas oportunamente…

Referências desta aula

- Esta aula foi elaborada com base na bibliografia e em pelos menos os seguintes materiais de apoio:
  - SEDGEWICK, Robert; WAYNE, Kevin. Algorithms, 4th Edition: Online Content. University of Princeton, 2024.
  - SHI, Min. Design & Analysis of Algorithms. CSCE500, University of Louisiana 2025.
  - BELL, Ana. Introduction to CS and Programming using Python. 6.100L, MIT. 2022.

Introdução a Algoritmos

O que é uma algoritmo?

- **Definições informais:**
  - Um procedimento definido, finito, que possui entradas e saídas (D. Knuth)
  - Uma sequência finita de passos bem definidos para resolver um **problema**, gerando dados de saída a partir dos dados de entrada
  - Problema: especifica a relação entre a entrada e a saída

- Com essa definição, uma receita culinária é um algoritmo!

Algoritmo da Panqueca

- Problema:
  - Criar uma panqueca com tamanho adequado para alimentar n pessoas
- Entradas (ingredientes):
  - Saco de farinha de trigo
  - Bandeja de ovos
  - Caixa de leite
  - Tabletes de de manteiga
  - Açúcar
  - Sal
  - Fermento
  - Bol
  - Frigideira
  - Concha
  - Garfo
  - Espátula
- Saídas:
  - 5*n panquecas de 10cm de diâmetro
- Algoritmo em pseudocódigo
  - Colocar o bol sobre a mesa
  - Depositar n*200g de farinha no bol
  - Quebrar os n ovos e despejar o conteúdo no bol
  - Depositar n*200ml de leite no bol
  - Liquefazer n*100g de manteiga e acrescentá-la ao bol
  - Acrescentar as 2n colheres de sal e de açúcar ao bol
  - Acrescentar as 2n colheres de fermento ao bol
  - Misturar **até que a massa fique homogênea**
  - Pré-aquecer a frigideira no fogão
  - **Repetir, até acabar a massa**
    - Retirar a massa do bol com a concha
    - Depositar a conchada sobre a frigideira
    - Aguardar surgirem bolhas sobre a massa
    - Utilizar a espátula para virar a massa
    - Aguardar o cozimento
    - Depositar a panqueca sobre o prato
  - Repartir as panquecas em n porções

Análise do Algoritmo

- Realização de operações básicas
  - O cozinheiro, ao contrário do computador, é extremamente versátil em suas habilidades
- Quanto tempo o algoritmo leva para executar?
- Do que depende esse tempo?
  - Tempo de execução x Complexidade

Computação, Algoritmos e Programas

- Apesar da perspectiva de uma boa panqueca, estamos interessados em manipular dados com nossos computadores (e outros dispositivos eletrônicos)

- Um **algoritmo** é um ente abstrato, que independe do computador em que está sendo executado, e é tipicamente descrito por fluxogramas, pseudocódigo ou mesmo em português
- Um **programa** é uma implementação desse algoritmo, através de uma **linguagem de programação**

Algoritmo euclidiano do máximo divisor comum (MDC)

- Entradas: a, b (números inteiros estritamente positivos)
- Saídas: mdc (número inteiro)

- Pseudocódigo:

```
mdc(a,b):

enquanto b > 0
  (a,b) ← (b, a mod b)
retornar a
```

Teste de Mesa (1/2)

- Consiste em “executar” o pseudocódigo para uma entrada em particular:
  - mdc(270, 210)
    - a=270, b=210
    - b > 0?
      - SIM
      - a=210, b=(270 mod 210)
      - a=210, b=60
    - b > 0?
      - SIM
      - a=60, b=(210 mod 60)
      - a=60, b=30

```
mdc(a,b):

enquanto b > 0
  (a,b) ← (b, a mod b)
retornar a
```

Teste de Mesa (2/2)

(...)
- a=60, b=30

- b > 0?
  - SIM
  - a=30, b=(60 mod 30)
  - a=30, b=0
- b > 0?
  - NÃO
- **Retornar a=30**

```
mdc(a,b):

enquanto b > 0
  (a,b) ← (b, a mod b)
retornar a
```

Complexidade e pior caso de execução

- A **complexidade de um algoritmo** é independente da máquina na qual ele executa
- A complexidade é intrínseca ao próprio algoritmo, e guarda proporção com o tamanho das entradas
- No caso deste algoritmo, o pior caso acontece quando temos dois números inteiros que aparecem sucessivamente na sequência de Fibonacci
  - Sequência de Fibonacci: primeiro número é 0, segundo número é 1, demais números são a soma dos dois últimos ⇒ {0, 1, 1, 2, 3, 5, 8, 13, 21, 34, …}
- A complexidade deste algoritmo, no pior caso, é descrita na notação do O-grande (big-O); pode-se demonstrar que, neste caso, ela será = **O(log b)**

Complexidades mais comuns (tempo assintótico)

- Tempo polinominal - $O(n^k)$
  - $O(n)$ - tempo linear
  - $O(n^2)$ - tempo quadrático
  - $O(n^3)$ - tempo cúbico
- Tempo logarítmico:
  - $O(\log n)$ - tempo logarítmico
- n log n
  - $O(n \log n)$
- Tempo não-polinominal:
  - $O(2^n)$
  - $O(n!)$ - tempo fatorial

Projetando bons algoritmos

- O objetivo de um bom algoritmo é resolver um problema dentro do menor tempo possível
  - Para pequenas instâncias do problema (pequenas entradas), talvez possa haver algoritmo com complexidade maior, mas que resulte em tempo de execução prático menor
- No caso geral, vamos trabalhar com dados de tamanho arbitrário
- Projetamos (ou escolhemos) o algoritmo, portanto, para que possua a mais baixa complexidade possível, ou seja, ter o menor tempo assintótico

Linguagens de Programação

- As linguagens de programação oferecem um conjunto básico de construtos ou primitivas: operações básicas e tipos de dados básicos
- As linguagens possibilitam a criação de operações complexas a partir desses construtos
- As linguagens seguem uma **gramática,** que define os aspectos **léxicos**, **sintáticos** e **semânticos**
  - O léxico diz respeito à forma de reconhecer os conjuntos de letras e símbolos utilizados no programa
  - A sintaxe diz respeito ao papel gramatical de cada “palavra” e separador (ex.: nome de um valor constante, símbolo que expressa um operador matemático, etc)
  - A semântica diz respeito ao significado do programa que foi implementado (ou seja, o algoritmo que ele de fato implementa)
- Há diferentes **paradigmas de programação**
  - Imperativo, Funcional, Lógico, Orientado a Objeto

Introdução à Linguagem Python

Python

- Linguagem multiparadigma, nascida nos anos 90
  - Procedural/imperativa, orientada a objetos, funcional, estruturada, …
- Criação de Guido van Rossum, o “ditador benevolente vitalício” da linguagem, agora emérito da função
- Tornou-se extremamente popular a partir dos anos 2000
- Teve a adesão crescente
- A implementação de referência é compilada para *bytecodes* de uma VM específica
- Python é **fracamente tipada** (duck typing) e **indentada**
- Está, atualmente, em sua versão 3.14

Tipos de dados primitivos em Python

- A linguagem conta com os seguintes tipos de dados **escalares** nativos**:**
  - **int:** representa números inteiros
  - **float:** representa números reais (em ponto flutuante)
  - **bool:** representa valores booleanos (verdadeiro/falso)
  - **str:** representa texto (strings)
  - **NoneType:** tipo nulo especial

- Também conta com os seguintes tipos de dados **vetoriais** nativos:
  - complex (números complexos), list (lista de elementos), tuple (tuplas), range (sequências), dict (hash ou mapa), set (conjunto), frozenset (conjunto ordenado), bytes (vetor de dados de 8 bits constante), bytearray (vetor de dados de 8 bits variável), memoryview*

Alguns operadores e funções pré-definidas em Python

- Operadores
  - Matemáticos: `+, -, *, /, %, **, //`
  - Booleanos: **`or, and, not`**
  - Comparadores: `<, <=, >, >=, ==, !=, is, is not`
  - Operadores *bitwise*: `|, ^, &, <<, >>, ~x`

- Python possui uma série de funções pré-definidas, tipicamente capazes de manipular os tipos primitivos escalares e vetoriais
- Algumas funções:
  - **print()**, input(), help(), exit()

Algumas das Estruturas de Controle de Fluxo

```
if condição:
    <...>
elif condição:
    <...>
else:
    <...>
```

```
for <item> in <sequência>:
    <...>
    [else: <...> ]
```

```
while condição:
    <...>
    [else: <...> ]
```

Instalação do interpretador/compilador

- Para Windows, a distribuição de referência da linguagem Python pode ser instalada através do “Python Install Manager”
  - Procurar na Windows Store
- Para as distribuições GNU/Linux, usualmente Python está disponível através do sistema de gerenciamento de pacotes
  - RedHat-based (RHEL, Rocky, Alma): dnf install python3
  - Debian-based (Debian, Ubuntu, PVE): apt install python

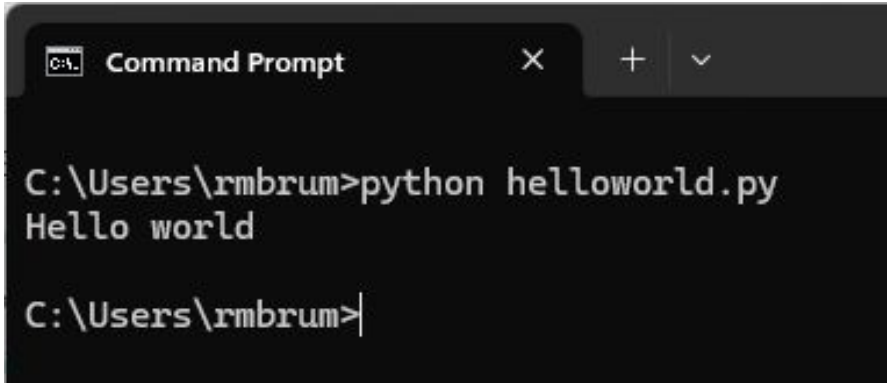
- Utilize esse momento para verificar se já possui uma distribuição de Python instalada em seu sistema, e instale-a se for o caso

Um primeiro programa - “Hello, World”

- A tradição no mundo da programação é a de iniciar com a construção de um programa “Hello, World” ou “Olá, mundo”

helloworld.py

```
print(“Hello world”)
```



Algoritmo do MDC

Vamos implementar o algoritmo do MDC em Python:

```
[Pseudocódigo mdc(a,b):]

enquanto b > 0
   (a,b) ← (b, a mod b)
retornar a
```

```
a, b = 270, 210

while b > 0:
    a, b = b, a % b
    print("Valores: ", a, b)
print("MDC: ", a)
```

Algoritmo do MDC

Vamos definir uma função:

```
def mdc(a,b):

    while b > 0:
        a, b = b, a % b
    return a
```

```
a, b = 270, 210

while b > 0:
    a, b = b, a % b
    print("Valores: ", a, b)
print("MDC: ", a)
```

Construção e execução de exemplos

- Vamos realizar alguns exemplos juntos?
  - Fonte: “30 days of Python”
  - Variables, Built-in Functions